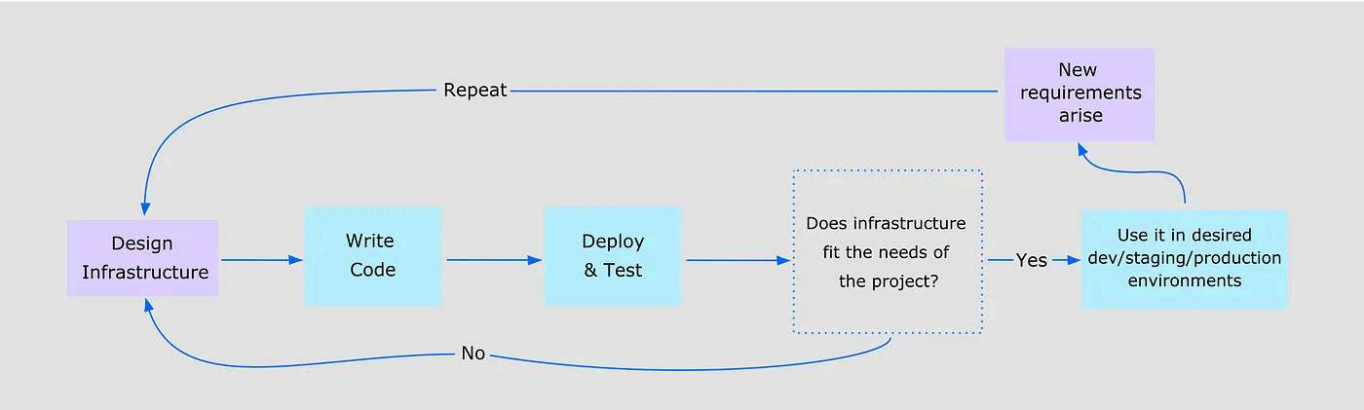
Understanding Infrastructure as Code (IaC)

Meriem Terki Follow 6 min read · Aug 18, 2024

14



Do you want to understand IaC in a clear and easy way , I am on a challenge to learn and get better at terraform in 30 days and I am exited to take you along with me in this journey .

This is day 1 and in this blog , we are going to see a broad overview about IaC and Terrafrom and why they are needed ?

Get Meriem Terki's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email Subscribe

Let's start !

What is Infrastructure as Code?

- Infrastructure as Code is the **practice of managing and provisioning computing infrastructure through machine-readable configuration files**, rather than through physical hardware configuration or interactive configuration tools. By treating infrastructure as code, teams can **apply software engineering practices like version control, automated testing, and continuous delivery to their infrastructure.**
- **IaC tools can be broadly categorized** into:

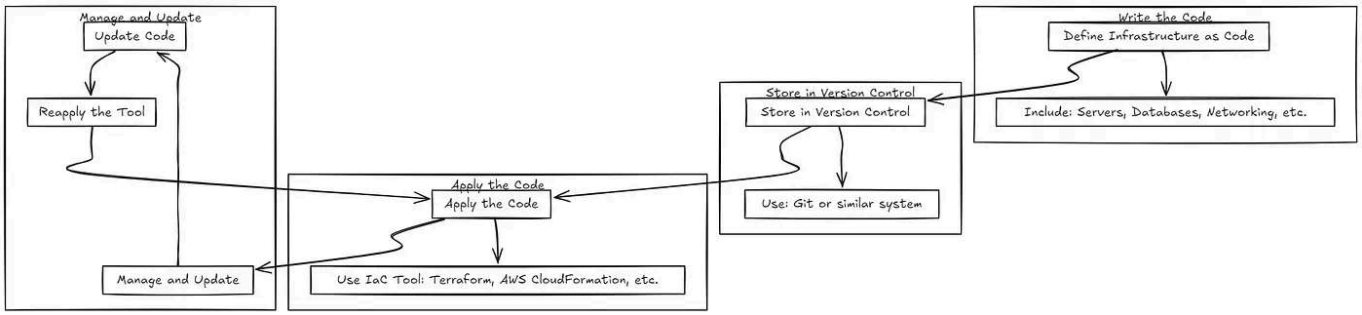
1. **Ad hoc Scripts:** Simple scripts that automate specific tasks but lack scalability and maintainability.
2. **Configuration Management Tools:** Tools like Chef, Puppet, and Ansible that manage the configuration of servers and ensure they are in a desired state.
3. **Server Templating Tools:** Tools like Docker, Packer, and Vagrant that create consistent environments for applications to run in, whether on-premises or in the cloud.
4. **Orchestration Tools:** Tools like Kubernetes that manage the deployment, scaling, and operation of containerized applications across clusters of hosts.

5. **Provisioning Tools:** Tools like Terraform, CloudFormation, and OpenStack Heat that automate the provisioning of infrastructure resources.

Among these, **provisioning tools like Terraform** stand out because they **allow teams to define, deploy, and manage all aspects of their infrastructure as code, making the process more efficient and less prone to error.**

How Does Infrastructure as Code Work?

1. **Write the Code**: You **define what your infrastructure should look like using code.** This could include servers, databases, networking, and more.
2. **Store in Version Control**: The code is saved in a version control system like Git. This allows you to **track changes and collaborate with your team.**
3. **Apply the Code:** You use an IaC tool like Terraform or AWS CloudFormation to **read the code and set up your infrastructure automatically.**
4. **Manage and Update**: Whenever you **need to change your infrastructure**, you **update the code and reapply it**, and the tool makes the necessary changes.



Benefits of Infrastructure as Code (IaC): Highlights

1. **Improved Version Control & Error Reduction**

- **Version Control**: Adopting IaC integrates seamlessly with version control systems like GitHub, providing an audit trail of changes. This makes it easy to **track, troubleshoot, and roll back to previous configurations if necessary.**
- **Reduced Human Errors**: Automating infrastructure setup with code **reduces the risk of manual errors, ensuring consistent and reliable deployments.**

2. **Enhanced Business Continuity & Knowledge Sharing**

- **Siloed Knowledge Reduction**: IaC ensures that infrastructure configuration details are accessible to all relevant team members, reducing reliance on individual knowledge and promoting continuity within the team.
- **Documentation & Efficiency**: Writing code for infrastructure documents the setup and makes it easier for team members to understand and modify, streamlining operations and reducing manual tasks.

3. **Increased Deployment Speed & Security**

- **Faster Deployments**: IaC significantly increases deployment speed by automating the provisioning and configuration of resources, which allows teams to quickly adapt and respond to business needs.
- **Security & Compliance**: IaC allows for the enforcement of security and compliance policies directly in the code, ensuring consistency across

environments without compromising governance.

4. Streamlined Operations & Resource Reusability

- **Reusability**: The use of variables in IaC code allows for the reuse of configurations across different environments, ensuring uniformity between development, QA, and production settings.
- **Automation Efficiency**: Automating tasks like VM provisioning saves time and resources, making it a valuable investment for organizations aiming for consistency and efficiency across environments.

5. Improved Collaboration & Reduced Miscommunication

- **Code Documentation**: Effective documentation in IaC improves collaboration by making infrastructure changes easier to understand, reducing the likelihood of miscommunication and enhancing operational continuity.
- **Testing & Validation**: IaC facilitates early detection of issues during the testing and validation stages, reducing the costs associated with fixing problems in production environments.

6. Disaster Recovery & Rapid Response

- **Robust Disaster Recovery**: IaC provides a quick and efficient way to redeploy resources in the event of failures or outages, minimizing downtime and ensuring business continuity.
- **Quick Rollbacks**: Version control integration with IaC allows for easy rollback to previous configurations, significantly speeding up the

restoration of services after critical failures.

7. Cost-Effective Development Cycles

- **Shifting Left**: Integrating testing tools directly into the development environment encourages early detection and resolution of issues, reducing costs associated with late-stage fixes.
- **Reduced Bottlenecks**: The speed and efficiency of IaC in deploying infrastructure resources help eliminate bottlenecks in manual processes, allowing for quicker response times to evolving business needs.

How Does Terraform Compare to Other IaC Tools?

When selecting an Infrastructure as Code tool, several factors should be considered:

1. **Configuration Management vs. Provisioning:**

- **Configuration management tools like Ansible** focus on **managing and maintaining the state of existing infrastructure.**
- **Provisioning tools like Terraform** are **designed to create and manage the underlying infrastructure itself.**

2. Mutable vs. Immutable Infrastructure:

- **Mutable infrastructure** allows changes to be made directly on running systems, which can lead to configuration drift.
- **Immutable infrastructure**, **preferred by tools like Terraform**, treats infrastructure components as disposable; any change requires a new

version of the infrastructure to be provisioned.

3. Procedural vs. Declarative Language:

- **Procedural tools** (e.g., **Chef**) require you to define step-by-step instructions for how to achieve the desired state.
- **Declarative tools** (e.g., **Terraform**) let you specify the desired end state, and the tool figures out how to achieve it.

4. General-Purpose vs. Domain-Specific Language:

- **General-purpose languages** offer more flexibility but can be complex.
- **Domain-specific languages (DSL)** like the HashiCorp Configuration Language (HCL) used by Terraform are easier to use and tailored to the specific needs of infrastructure management.

5. Master vs. Masterless Architecture:

- Some tools require a master server to coordinate the deployment (e.g., Puppet), while others, like Terraform, do not.

6. Agent vs. Agentless:

- Agent-based tools require software to be installed on the managed servers, while agentless tools (like Terraform) do not, making them easier to use in diverse environments.

7. Paid vs. Free Offering:

- The cost of the tool and its features, especially in a production environment, is a critical consideration. Terraform offers an open-source version with a robust set of features and an enterprise version for more advanced needs.

8. Community Size and Maturity:

- The size of the community and the maturity of the tool can impact how easily you can find support, plugins, and resources.

Why Terraform Stands Out ?

- **Terraform's declarative approach to infrastructure management,** combined with **its support for a wide range of cloud providers** and **its large, active community**, makes it a powerful tool for DevOps teams. It enables the **consistent, repeatable, and auditable management of infrastructure, making it easier to maintain complex environments**.
- By using Terraform, **teams can automate the provisioning of infrastructure, reduce the risk of human error, and improve the efficiency of their DevOps processes**. This leads to **faster, more reliable software delivery, with infrastructure that is easier to scale and maintain.**

Conclusion

As organizations continue to embrace DevOps practices, the need for efficient, scalable, and reliable infrastructure management tools becomes increasingly important. Terraform, with its infrastructure-as-code approach, is a key enabler of modern DevOps practices, helping teams to deliver software more quickly, more reliably, and with greater confidence.

Resources Used :

- https://www.terraformupandrunning.com/
- https://www.youtube.com/watch?v=zWw2wuiKd5o
- https://www.youtube.com/watch?v=u3fgc4mcydM&t=45s

That's all I have for today folks. Thank you for reading and/or following along! I hope this blog was helpful and worth your while. Stay tuned for my next project on this journey into the cloud.

Let's connect on LinkedIn! 👉 https://www.linkedin.com/in/meriemterki/

Infrastructure As Code | Terraform | DevOps | Automation

Written by Meriem Terki

471 followers · 1.1K following

Data, Cloud & AI enthusiast| Follow me on my journey

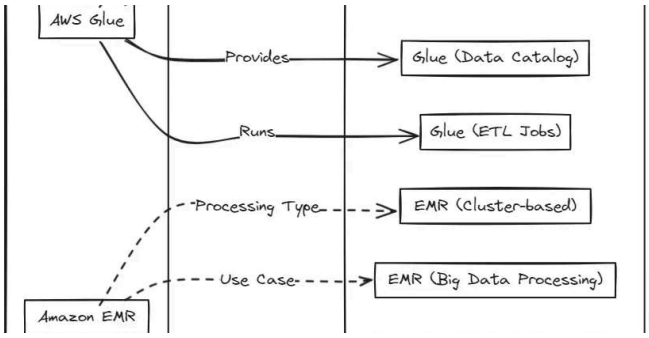
Follow

No responses yet

Write a response

What are your thoughts?

More from Meriem Terki



Meriem Terki

AWS Glue vs. Amazon EMR: Choosing the Right Tool for Your...

Introduction

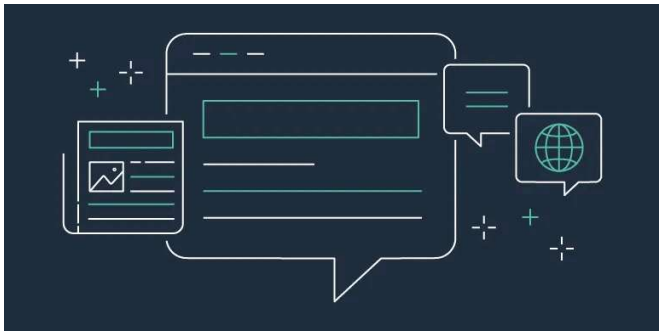
May 28, 2024 4 1

Meriem Terki

Step-by-Step Guide to Setting Up Terraform, AWS CLI, and Your AW...

We are moving forward in this 30 day terraform challenge and this is day 2 .

Aug 20, 2024 17



Meriem Terki

Understanding Dual-Layer Server-Side Encryption with AWS KMS...

What is DSSE-KMS?

Meriem Terki

How to Manage Terraform State: A Step-by-Step Guide

We are moving in the 30 days terraform challenge and this is day 6.

Jun 4, 2024 52

Aug 27, 2024 52

See all from Meriem Terki

Recommended from Medium

In Write A Catalyst by DevOps voice

🧰 COMPLETE DEVOPS CHEAT SHEET HANDBOOK

A practical, example-driven DevOps command reference covering Linux, Git,...

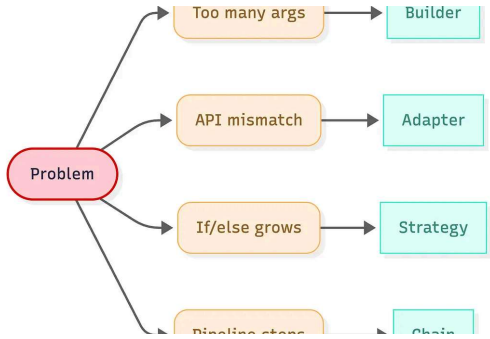
Jan 14 384 3

In CodeX by MayhemCode

Why Thousands Are Buying Mac Minis to Escape Issues with Big...

Something strange happened in early 2026. Apple stores started running low on Mac...

Feb 15 3.5K 57



In Women in Technology by Alina Kovtun ✨

Stop Memorizing Design Patterns: Use This Decision Tree Instead

Choose design patterns based on pain points: apply the right pattern with minimal over-...

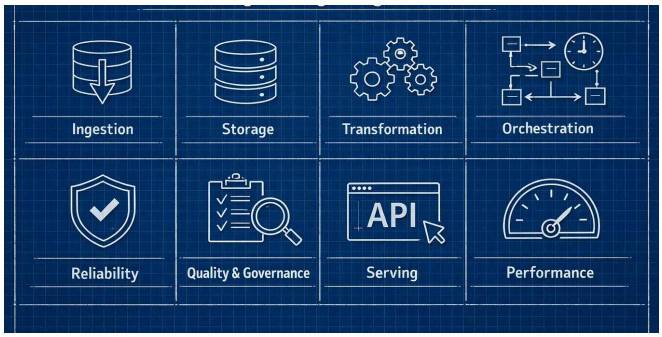
Jan 29 5.1K 43

Bhavyansh

The CI/CD Setup That Cut Our Deployment Time from 40 Minute...

How we optimized our deployment pipeline and got back 32 minutes of our lives (per...

Oct 27, 2025 254 7

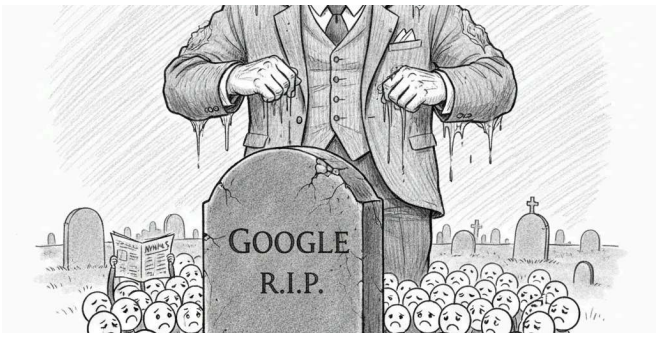


In AWS in Plain English by Khushbu Shah

Data Engineering Design Patterns You Must Learn in 2026

These are the 8 data engineering design patterns every modern data stack is built on....

Jan 5 1.1K 25



In Level Up Coding by Teja Kusireddy

Google Is Quietly Dismantling Everything OpenAI Built

The most dangerous failure in Silicon Valley isn't bankruptcy. It's becoming the engine...

Feb 17 4K 133

See more recommendations